

KPC Order-to-Cash Leakage - Problem-Framing Memo

Inuka Hackathon - Stage 1 (Data Engineering) - Problem 7D, Domain D: Revenue Assurance, Billing & Reconciliation
Author: Lameck Irungu | Date: 24 July 2026

The problem

KPC's order-to-cash cycle runs through four handoffs -- depot loading, gate dispatch, finance invoicing, and AR payment -- each owned by a different system and, today, reconciled manually if at all. Every handoff is a place product or revenue can go missing without anyone noticing until a periodic audit, weeks later, tries to explain a gap.

Why it matters now

This is a revenue-assurance problem, not a data-quality nicety. Reconciliation gaps compound in two directions: lost cash (shipped and never billed, or billed and never collected) and lost audit trail (KPC cannot show a regulator or lender exactly where a shilling of expected revenue went). Closing it needs the same discipline telecom and utility revenue-assurance teams apply: automated, continuous matching across every handoff, not a spot-check.

What we built (Stage 1 scope)

An automated ETL + reconciliation pipeline that ingests four messy, independently-owned exports (mixed date formats, currency-formatted numbers, depot-name aliasing, missing fields); cleans and validates every table against explicit schemas with a data-quality gate that fails hard on structural defects but never on the business gaps the pipeline exists to find; and reconciles the full loading -> dispatch -> invoice -> payment chain, attributing every leaked shilling to exactly one of five named categories so nothing is double-counted:

Category	What it means
Dispatch capture gap	Loaded, but no dispatch record exists at all
Shrinkage	Dispatched volume is materially below loaded volume
Billing gap	Confirmed dispatch, but never invoiced
Underbilling	Invoiced for less volume than was dispatched
Collections gap	Invoiced correctly, but payment is partial or outstanding

It ships evidence, not just code: 19 automated tests (unit + integration), a CI workflow that lints, tests, and re-runs the quality gate on every push, and a containerized executive dashboard with AI-generated audit narratives and an ROI model.

Early findings

Run against a synthetic dataset calibrated to plausible KPC depot volumes and ex-depot pricing (45 days, 6 depots, 3 products, 20 customers -- not live KPC data): 5.5% of expected ex-depot revenue leaks before it becomes collected cash -- KES 689.1M over the window, ~KES 5.59B annualized. 10.3% of loadings (290 of 2826) carry at least one reconciliation exception. Leakage concentrates: Kisii depot and Baraka Downstream are the single largest depot- and customer-level exposures respectively -- a few targeted fixes close most of the gap, not a company-wide program.

What's next

Stage 2: connect to live KPC extracts; add statistical diagnostics and a predictive shrinkage/collections-risk model on top of this reconciliation foundation. Stage 3: harden into the monitored, CI/CD-deployed service this pipeline already points at, and validate the ROI model's assumptions against real recovery data.

All KES figures in this memo and the dashboard are illustrative -- computed on synthetic data, not KPC's live systems. Every claim above is reproducible: `make pipeline` regenerates every number from the committed source data.